Masterclass: pandas, matplotlib & seaborn

Nov 9, 2026 · @xshel

How to use this course

This course teaches every pandas, matplotlib and seaborn feature that `clean.py` and `analysis.py` use, from zero. Each module explains one idea, shows a tiny example you can run, then points to the exact lines in your scripts that use it.

Who it's for: someone who knows basic Python (variables, `if`, functions) but has never used pandas.

How long: about 8–10 hours if you type every example yourself. Typing beats reading: you'll remember far more.

How each module works:

1. **Idea**: what the tool is and why it exists.
2. **Example**: a small piece of code with its output.
3. **In your scripts**: the real lines from `clean.py` (C) or `analysis.py` (A) that use it.
4. **Try it**: a short exercise. Answers are in Module 10 hints.

Setup. Open a terminal in your project folder and start an interactive Python session, or better, a Jupyter notebook so you can see tables nicely:

```
.venv/bin/pip install pandas matplotlib seaborn jupyter
.venv/bin/jupyter notebook
```

Every example starts with these imports:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Course map:

Module	Topic	Used mostly in
0	Python essentials	both
1	pandas foundations	both

Module	Topic	Used mostly in
2	Loading and saving data	both
3	Selecting and filtering	both
4	Cleaning data	clean.py
5	Dates and time series	both
6	Summarizing and grouping	analysis.py
7	matplotlib foundations	analysis.py
8	matplotlib customization	analysis.py
9	seaborn	analysis.py
10	Capstone exercises	both
—	Cheat sheet	—

Module 0: Python essentials used in the scripts

Your scripts use eight plain-Python tricks again and again. Learn these first and the pandas code becomes much easier to read.

0.1 f-strings and format specs

An f-string puts variables inside text. Write `f` before the quotes and put the variable in `{}`. After a colon you can add a **format spec** that controls how the value looks.

```
n = 3500000
share = 0.6234
print(f"{n}")        # 3500000
print(f"{n:,}")      # 3,500,000      , = thousands separator
print(f"{share:.1f}") # 0.6           .1f = 1 decimal
print(f"{share:.1%}") # 62.3%        % = times 100, add %
print(f"{n/1e6:.1f}M") # 3.5M        1e6 = 1,000,000
```

Spec	Meaning	1234567.891 becomes
`:,`	thousands separator	1,234,567.891

Spec	Meaning	1234567.891 becomes
`:.0f`	no decimals	1234568
`:,.0f`	both	1,234,568
`:.1%`	percent, 1 decimal	123456789.1%

Dates have their own codes: `f"{date:%b %Y}"` gives `Mar 2024` (`%b` short month, `%Y` year, `%m` month number, `%d` day).

In your scripts: C line 105 `{total_in:,}`, A line 35 (`millions` formatter), A line 82 `{peak:%b %Y}`, A lines 160–162 `:.1%`.

The older `.format()` method does the same thing: `"{:,.0f}".format(v)` (A line 48).

0.2 Dictionaries

A dictionary maps **keys** to **values**. pandas uses them everywhere to say "this column → that setting".

```
rename = {"Product": "product", "State": "state"}
rename["Product"]          # 'product'

a = {"x": 1}
b = {"y": 2}
a | b                      # {'x': 1, 'y': 2}   merge (Python 3.9+)
```

In your scripts: `PRODUCT_MAP`, `SHORT`, `RENAME` (C lines 23–51), the `dtype=` dictionary (A lines 61–62).

0.3 Loops, enumerate and zip

```
for col in ["product", "state"]:
    print(col)

for i, name in enumerate(["a", "b", "c"], 1):   # numbering starts at 1
    print(i, name)          # 1 a / 2 b / 3 c

for name, color in zip(["x", "y"], ["red", "blue"]):  # pairs items up
    print(name, color)      # x red / y blue

dict(zip(["x", "y"], ["red", "blue"]))   # {'x': 'red', 'y': 'blue'}
```

In your scripts: C line 92 `enumerate(reader, 1)`, A line 47 `enumerate(s.values)`, A line 143 `dict(zip(...))`.

0.4 Tuples and unpacking

A tuple is a fixed group of values in parentheses. You can **unpack** it into several variables at once.

```
size = (9, 4.5)
width, height = size
fig, ax = plt.subplots()   # a function returning two things
```

0.5 Comprehensions

A comprehension builds a list or dict with a loop in one line.

```
[x * 2 for x in [1, 2, 3]]               # [2, 4, 6]
{c: "category" for c in ["a", "b"]}     # {'a': 'category', 'b': 'category'}
```

In your scripts: A line 61.

0.6 lambda

A `lambda` is a tiny nameless function, written on one line. Use it when a library wants a function and it's too small to deserve a `def`.

```
double = lambda x: x * 2
double(5)          # 10

# same as:
def double(x):
    return x * 2
```

In your scripts: A line 35. `_` is the conventional name for an argument you receive but ignore.

0.7 Conditional expression and None

```
label = "big" if n > 1000 else "small"   # if/else in one line
top_n = None                              # "no value"
if top_n:                                 # None counts as False
    ...
```

0.8 Booleans are numbers

`True` behaves like 1 and `False` like 0. So the **average of True/False values is a percentage**. This is one of the most useful tricks in data analysis.

```
values = [True, True, False, True]
sum(values)                  # 3
sum(values) / len(values)    # 0.75 → 75% are True
```

In your scripts: A lines 129, 160–162.

Try it (0): Print `1234.6` as `1,235`. Then build a dict `{"a": 1, "b": 2, "c": 3}` with `zip`.

Module 1: pandas foundations

pandas gives Python a **table** object called a DataFrame. Everything else in pandas is a way to load, change, filter, summarize or save that table.

1.1 DataFrame and Series

- A **DataFrame** is the whole table: rows and named columns.
- A **Series** is one column: a list of values with labels.

```
df = pd.DataFrame({
    "company": ["Chase", "Equifax", "Chase", "Citi"],
    "state":   ["NY", "GA", "CA", "NY"],
    "timely":  [True, False, True, True],
})
print(df)
```

```
   company state  timely
0    Chase    NY    True
1  Equifax    GA   False
2    Chase    CA    True
3     Citi    NY    True
```

```
df["company"]          # one column → a Series
df[["company", "state"]]  # a list of columns → a smaller DataFrame (note the
double brackets)
```

1.2 The index

The numbers on the left (0, 1, 2, 3) are the **index**: a label for each row. By default it's just row numbers, but it can be anything: dates, names, products.

The index matters for three things in your scripts:

- `value_counts()` returns a Series whose **index holds the names** and whose **values hold the counts**. `barh` uses `s.index` for bar labels and `s.values` for bar lengths (A line 42).
- `set_index("date_received")` makes dates the row labels so `resample` can group by time (A line 74).
- `reset_index()` turns the index back into a normal column (A line 141). `to_csv(index=False)` skips writing it (C line 102).

1.3 Shape, dtypes and a first look

```
df.shape      # (4, 3) → 4 rows, 3 columns
df.dtypes     # the type of each column
df.head()     # first 5 rows
df.columns    # the column names
len(df)       # number of rows
```

Each column has one **dtype** (data type). The ones you'll meet:

dtype	Holds	Missing value allowed?
`object` / `str`	text	yes (NaN)
`int64`, `int16`	whole numbers	**no**

dtype	Holds	Missing value allowed?
`Int64`, `Int16` (capital I)	whole numbers	yes (`<NA>`)
`float64`	decimals	yes (NaN)
`bool`	True/False	no
`datetime64`	dates and times	yes (NaT)
`category`	a few repeated text values	yes

The number after `int` is the size in bits: `int16` uses 2 bytes and holds −32,768 to 32,767; `int64` uses 8 bytes. Smaller types save memory on big data.

In your scripts: A lines 64–65 print `shape` and `dtypes` to check the load worked.

1.4 Vectorized operations: no loops

The single most important idea in pandas: **you operate on a whole column at once**, never row by row.

```
prices = pd.Series([10, 20, 30])
prices * 2          # 20, 40, 60
prices > 15         # False, True, True   ← a column of booleans
prices.mean()       # 20.0
```

This is both shorter and about 100× faster than a Python `for` loop over rows. If you catch yourself writing `for row in df...`, there's almost always a vectorized way.

1.5 Accessors: .str and .dt

Some tools only make sense for one kind of column. pandas groups them behind an **accessor**:

- `.str` gives text tools: `df["company"].str.upper()`, `.str.strip()`, `.str.contains("bank")`.
- `.dt` gives date tools: `df["date"].dt.year`, `.dt.month`, `.dt.days` (on a time difference).

```
df["company"].str.upper()   # CHASE, EQUIFAX, CHASE, CITI
```

In your scripts: C lines 65, 68–69 (`.str`), C lines 78–79 and A line 71 (`.dt`).

1.6 Method chaining

Most pandas methods return a new table or Series, so you can chain them:

```
df["company"].value_counts().head(10)
```

Read it left to right: take the column → count values → keep the first 10. Long chains can be split over lines by wrapping them in parentheses (A lines 139–141).

Also note that most methods **don't change the original**; they return a new object. That's why your scripts always reassign: `df = df.dropna(...)`.

Try it (1): Build the small `df` above. Print the share of `timely` rows using one method call.

Module 2: Loading and saving data

`pd.read_csv` turns a CSV file into a DataFrame, and `df.to_csv` writes one back. Their options decide how much memory you use and whether your types survive the trip.

2.1 read_csv: the options you use

```
df = pd.read_csv("complaints.csv")   # simplest form: load everything, guess types
```

Option	What it does	Used in
`usecols=[...]`	load only these columns; the rest are never read	C line 91
`dtype=str`	read every column as text (no guessing)	C line 91
`dtype={"col": type}`	set the type of each column	A lines 61–62
`parse_dates=[...]`	read these columns as real dates	A line 60
`chunksize=N`	return the file N rows at a time instead of all at once	C line 91

Why `dtype=str` in clean.py? If pandas guesses types, a column can be guessed differently in each chunk (numbers in chunk 1, text in chunk 2). Reading everything as text is predictable. The cleaning function then converts each column on purpose.

2.2 Reading in chunks

A 5.7 GB file doesn't fit in memory. With `chunksize`, `read_csv` returns a **reader** you loop over:

```
reader = pd.read_csv("big.csv", chunksize=1_000_000)
for chunk in reader:
    print(len(chunk))   # each chunk is a normal DataFrame of up to 1M rows
```

The rule of chunked work: **anything that must be remembered across chunks lives outside the loop.** In clean.py that's the counters and `seen_ids` (C lines 85–86).

2.3 The category dtype

A column like `product` has ~15 different values repeated millions of times. `category` stores each text **once** and gives every row a small number code.

```
s = pd.Series(["Mortgage", "Credit card", "Mortgage"] * 1_000_000)
s.memory_usage(deep=True) / 1e6                       # ~190 MB
s.astype("category").memory_usage(deep=True) / 1e6     # ~3 MB
```

Use it for text columns with **few unique values** (product, state, channel). Don't use it for columns where almost every value is different (like an ID or a free-text description).

One side effect: a category column remembers **all possible values, even ones no longer present** after filtering. That's why A line 129 uses `observed=True` and A line 142 converts back with `.astype(str)`.

2.4 Checking memory

```
df.memory_usage(deep=True).sum() / 1e6   # total MB
```

`deep=True` measures text accurately. Without it, text columns are badly underestimated. (A line 66.)

2.5 to_csv: saving

```
df.to_csv("out.csv", index=False)
```

Option	What it does	Default
`index=False`	don't write row numbers as an extra column	writes them
`mode="a"`	**append** to the end of the file	`"w"` = overwrite
`header=True/False`	write the column-name line or not	True
`date_format="%Y-%m-%d"`	how dates are written	full timestamp

Writing chunks into one file needs all three together (C lines 102–103): append mode so chunks stack up, a header only on the first chunk, and deleting the old file before starting (C lines 88–89) so a rerun doesn't double everything.

2.6 CSV forgets types

CSV is plain text. Dates, `Int16`, `category`, `bool`: all of it becomes text when saved. That's why analysis.py has to pass `parse_dates` and `dtype` again when loading the cleaned file. (Formats like Parquet keep types and are much faster, a good next step once you're comfortable: `df.to_parquet("x.parquet")`.)

Try it (2): Load only the `Product` and `State` columns of the raw file, first 1,000 rows (`nrows=1000`). Compare memory before and after `astype("category")`.

Module 3: Selecting and filtering

Filtering in pandas always works the same way: build a **column of True/False** (a "mask"), then use it to keep the True rows.

All examples in this module use this table:

```
df = pd.DataFrame({
    "company": ["CHASE", "EQUIFAX", "CHASE", "CITI", "EXPERIAN"],
    "state":   ["NY", "GA", "CA", "NY", "CA"],
    "days":    [1, 0, 5, 2, 0],
})
```

3.1 Selecting columns

```
df["company"]             # one column (Series)
df[["company", "days"]]   # several columns (DataFrame)
```

3.2 Boolean masks

A comparison on a column gives one True/False per row:

```
df["state"] == "NY"    # True, False, False, True, False
```

Put that mask inside `df[ ]` to keep only the True rows:

```
df[df["state"] == "NY"]
```

```
  company state  days
0   CHASE    NY     1
3    CITI    NY     2
```

3.3 Combining conditions: &, |, ~

Symbol	Meaning	Example
`&`	and	`(df["state"] == "NY") & (df["days"] > 1)`
`|`	or	`(df["state"] == "NY") | (df["state"] == "CA")`
`~`	not (flips True/False)	`~(df["state"] == "NY")`

Always put each condition in parentheses. Python's `and` / `or` / `not` don't work on columns; use the symbols.

3.4 isin: "is one of these"

Instead of chaining many `|`, use `isin` with a list:

```
bureaus = ["EQUIFAX", "EXPERIAN"]
df[df["company"].isin(bureaus)]     # only bureau rows
df[~df["company"].isin(bureaus)]    # everything except bureaus
```

`isin` also accepts a set, which is very fast for big lookups.

In your scripts: C line 98 (`~isin(seen_ids)` removes IDs already written), A line 100 (excluding bureaus), A line 139 (keeping top 4 products), A line 161 (share of bureau complaints).

3.5 loc: rows and columns together

`df.loc[rows, columns]` selects both at once. Rows can be a mask; columns can be a name or a list.

```
df.loc[df["days"] > 1, "company"]            # companies with days > 1 → Series
df.loc[df["days"] > 1, ["company", "state"]] # → DataFrame
```

In your scripts: A line 100 `df.loc[~df["company"].isin(bureaus), "company"]`.

`loc` is also the safe way to **change** some rows:

```
df.loc[df["days"] == 0, "days"] = 1   # set 0 to 1, only where it was 0
```

(`iloc` is the position-based twin: `df.iloc[0]` is the first row, `df.iloc[:3]` the first three.)

3.6 head and tail

```
df.head(3)   # first 3 rows
df.tail(2)   # last 2 rows
```

After sorting or counting, `head(10)` means "top 10" (A lines 94, 100, 105).

3.7 Filtering by comparing to something computed

The condition can use any calculation, as long as it produces one True/False per row:

```
df[df["days"] > df["days"].mean()]     # rows above average
```

A line 71 does exactly this with months: it keeps rows whose month is `<=` the last full month.

Try it (3): From the table above, keep the rows in NY or CA where `days` is 0, using `isin` and `&`.

Module 4: Cleaning data

Cleaning means making the data **consistent** (same thing always spelled the same way), **typed** (dates are dates, numbers are numbers) and **complete enough** (useless rows removed, blanks labeled). clean.py does all three.

Examples use this messy table:

```
raw = pd.DataFrame({
    "Date received": ["2023-05-01", "banana", "2023-05-03", "2023-05-03"],
    "Company":       ["  Chase", "equifax", None, "Chase "],
    "Product":       ["Credit reporting", "Payday loan", "Mortgage", "Credit
reporting"],
    "Complaint ID":  ["101", "102", "103", "101"],
    "Timely?":       ["Yes", "No", "Yes", "Yes"],
})
```

4.1 Rename columns

```
df = raw.rename(columns={"Date received": "date_received", "Company":
"company",
                         "Product": "product", "Complaint ID":
"complaint_id",
                         "Timely?": "timely"})
```

Short names without spaces are easier to type and can't be mistyped as easily. Columns not in the dict keep their name. (C line 55.)

4.2 Convert types safely with errors="coerce"

```
df["date_received"] = pd.to_datetime(df["date_received"], errors="coerce")
# 2023-05-01, NaT, 2023-05-03, 2023-05-03    ← "banana" became NaT

df["complaint_id"] = pd.to_numeric(df["complaint_id"], errors="coerce")
```

`errors=`	What happens to a bad value
`"raise"` (default)	the whole script crashes
`"coerce"`	it becomes missing (NaT / NaN) and you deal with it next

On real-world data, `coerce` is almost always what you want. (C lines 58–59, 80.)

4.3 Missing values: find, drop, fill

```
df.isna().sum()                    # how many missing per column
df = df.dropna(subset=["date_received", "company"])   # drop rows missing
these
df["company"] = df["company"].fillna("Unknown")       # or label the blanks
```

The decision rule clean.py follows:

- **Drop the row** when the missing field makes it useless (no date, no product, no company: C line 60).
- **Fill with a label** when the row is still useful without it (`sub_issue` → "Unknown", `tags` → "None": C lines 70–72).

Without `subset=`, `dropna()` drops a row if **any** column is missing, which usually deletes far too much.

4.4 Text cleanup with .str

```
df["company"] = df["company"].str.strip()   # "  Chase" → "Chase"
df["company"] = df["company"].str.upper()   # "Chase" → "CHASE"
```

Without this, `"Chase"`, `" Chase"` and `"CHASE"` count as three different companies. Other useful ones: `.str.lower()`, `.str.title()`, `.str.replace("Inc.", "")`, `.str.contains("BANK")`. Missing values pass through `.str` methods untouched. (C lines 63–69.)

4.5 Replace values with a dictionary

```
PRODUCT_MAP = {"Payday loan": "Payday / personal loan"}
df["product"] = df["product"].replace(PRODUCT_MAP)
```

Values found in the dict are swapped; all others stay as they are. You can chain two replaces, as C line 67 does: first old name → current name, then long name → short label.

This is how you **merge categories** that mean the same thing, which matters a lot for counting: otherwise one product is split across several names.

4.6 Yes/No to True/False

```
df["timely"] = df["timely"].eq("Yes")   # same as df["timely"] == "Yes"
```

Anything that isn't exactly "Yes" (including blanks) becomes False. (C line 75.)

4.7 Create new columns

Assign to a name that doesn't exist yet:

```
df["year"] = df["date_received"].dt.year
df["is_bureau"] = df["company"].isin(["EQUIFAX", "EXPERIAN"])
```

(C lines 78–80.)

4.8 Nullable integers: Int16 vs int16

Plain integer types **cannot hold a missing value**. If a column of whole numbers has a gap, pandas silently turns it into decimals:

```
pd.Series([3, None, 7])                    # 3.0, NaN, 7.0   (float64)
pd.Series([3, None, 7]).astype("Int16")    # 3, <NA>, 7      (whole numbers
kept)
pd.Series([3, None, 7]).astype("int16")    # ERROR: cannot convert NaN
```

Rule of thumb: **capital `I` when blanks are possible**, lowercase when you're sure there are none. C line 78 uses `Int16` (date_sent can be missing); C line 79 uses `int16` (rows without a date were already dropped).

4.9 Duplicates

```
df.duplicated(subset="complaint_id")          # True for 2nd, 3rd… copies
df = df.drop_duplicates(subset="complaint_id") # keep the first copy only
```

`subset=` says which columns define "the same". Without it, rows must match on **every** column. Add `keep="last"` to keep the last copy instead.

Watch out: missing values count as equal to each other here, so rows with a missing ID are all treated as one duplicate group.

4.10 Duplicates across chunks

`drop_duplicates` only sees the current chunk. To catch a duplicate whose first copy was in an earlier chunk, remember what you've written in a set that lives **outside** the loop:

```
seen_ids = set()
for chunk in reader:
    chunk = chunk.drop_duplicates(subset="complaint_id")   # inside this
chunk
    chunk = chunk[~chunk["complaint_id"].isin(seen_ids)]   # vs. earlier
chunks
    seen_ids.update(chunk["complaint_id"].dropna())        # remember these
```

This is exactly C lines 86 and 97–99.

4.11 Writing cleaning as a function

clean.py puts every step in one function, `clean_chunk(df)`, that takes a table and returns the cleaned table. Benefits: you can test it on a small sample before running it on 5.7 GB, and the main loop stays short and readable.

Try it (4): Run every step of this module on `raw`. You should end with **1 row**: CHASE, ID 101. Row 102 goes because its date was "banana", row 103 because its company is missing, and the second 101 is a duplicate. Print the table after each step to see which row each step removes.

Module 5: Dates and time series

Once a column is a real date (`datetime64`), pandas can do arithmetic on it, pull out parts of it, and group rows by day, month or year.

5.1 Making dates

```
s = pd.to_datetime(pd.Series(["2024-03-15", "2024-03-20", "2024-04-02"]))
```

Or directly when loading: `pd.read_csv(..., parse_dates=["date_received"])` (A line 60).

5.2 Pulling out parts with .dt

```
s.dt.year        # 2024, 2024, 2024
s.dt.month       # 3, 3, 4
s.dt.day_name()  # Friday, Wednesday, Tuesday
```

(C line 79 uses `.dt.year`.)

5.3 Date arithmetic: timedelta

Subtracting two dates gives a **timedelta** (a duration). `.dt.days` turns it into a whole number of days.

```
sent = pd.to_datetime(pd.Series(["2024-03-18", None]))
recv = pd.to_datetime(pd.Series(["2024-03-15", "2024-03-20"]))
(sent - recv)                              # 3 days, NaT
(sent - recv).dt.days                      # 3.0, NaN   ← float because of
the gap
(sent - recv).dt.days.astype("Int16")      # 3, <NA>
```

This is exactly C line 78. The `Int16` step is explained in Module 4.8.

5.4 Periods: "which month is this?"

A **period** is a span of time, like "March 2024", rather than a single moment. `to_period("M")` turns each date into its month.

```
s.dt.to_period("M")                 # 2024-03, 2024-03, 2024-04
pd.Timestamp("2026-11-09").to_period("M") - 1   # Period('2026-10', 'M')
```

You can subtract from a period (`- 1` = previous month) and compare periods with `<=`. That's how A lines 70–71 drop the unfinished current month:

```
last_full_month = df["date_received"].max().to_period("M") - 1
df = df[df["date_received"].dt.to_period("M") <= last_full_month]
```

Note the difference: on a whole column you need `.dt.to_period`, while on a single date (`.max()` returns one value) you call `.to_period` directly.

Code	Period
`"D"`	day

Code	Period
`"W"`	week
`"M"`	month
`"Q"`	quarter
`"Y"`	year

5.5 resample: group rows by time

`resample` is like `groupby` (Module 6) but for time. It needs the dates **in the index**:

```
monthly = df.set_index("date_received").resample("MS").size()
```

Step by step:

1. `set_index("date_received")`: dates become the row labels.
2. `resample("MS")`: rows go into monthly buckets, each labeled with the first day of the month (`MS` = Month Start).
3. `.size()`: count rows per bucket.

The result is a Series: index = months, values = counts. Empty months get 0, so there are no gaps in a line chart.

Other frequencies: `"D"` daily, `"W"` weekly, `"QS"` quarter start, `"YS"` year start. Instead of `.size()` you can use `.sum()`, `.mean()` etc. on a column: `df.set_index("date_received").resample("MS")["days_to_company"].mean()`.

5.6 Finding where the maximum is: idxmax

```
monthly.max()      # the biggest count, e.g. 450000
monthly.idxmax()   # the LABEL where it happens, e.g. Timestamp('2024-03-01')
```

`idxmin()` is the opposite. A line 81 uses `idxmax` to place the "Peak" label on the chart.

5.7 Formatting dates for display

```
f"{monthly.idxmax():%b %Y}"     # 'Mar 2024'
df.to_csv(..., date_format="%Y-%m-%d")
```

Try it (5): Using the cleaned data, find the weekday with the most complaints (`.dt.day_name()` + `value_counts()`), and the year with the slowest median `days_to_company`.

Module 6: Summarizing and grouping

This is where analysis happens: turning millions of rows into a few numbers that answer a question. Nearly every chart in analysis.py starts with one of the tools below.

Examples use this table:

```
df = pd.DataFrame({
    "product":  ["Card", "Card", "Mortgage", "Card", "Mortgage", "Loan"],
    "response": ["Explained", "Money", "Explained", "Explained", "Explained",
"Money"],
    "timely":   [True, True, False, True, True, False],
    "days":     [1, 3, 10, 2, 4, 0],
    "year":     [2023, 2024, 2023, 2024, 2024, 2024],
})
```

6.1 Quick summaries

```
df["days"].mean()      # 3.33
df["days"].median()    # 2.5   middle value, ignores extreme outliers
df["days"].max()       # 10
df["product"].nunique()  # 3   number of different values
```

Median vs mean: one complaint taking 400 days pulls the mean up a lot but barely moves the median. For "typical" durations, prefer the median (A line 163).

6.2 describe

```
df.describe()                   # stats for number columns
df.describe(include="all").T    # all columns, flipped so each column is a
row
```

For text columns it reports `count` (non-empty), `unique` (different values) and `top` (most frequent). A line 67 keeps just those three. `.T` (transpose) swaps rows and columns, which is easier to read when you have many columns.

6.3 value_counts: the workhorse

```
df["product"].value_counts()
```

```
Card        3
Mortgage    2
Loan        1
```

Sorted biggest first, names in the index, counts as values: exactly what a bar chart needs. Add `normalize=True` for shares instead of counts (`Card 0.5`…). Add `.head(10)` for a top 10.

In your scripts: A lines 90, 94, 100, 105, 109, 134, 138.

6.4 groupby: split, apply, combine

`groupby` answers "for each X, what is Y?". It works in three steps:

1. **Split** the rows into groups (one per product).
2. **Apply** a calculation to each group (the mean of `days`).
3. **Combine** the results into one Series.

```
df.groupby("product")["days"].mean()
```

```
product
Card        2.0
Loan        0.0
Mortgage    7.0
```

Read it as: group by product → take the `days` column → average it.

Common calculations after `groupby`:

Method	Gives
`.size()`	number of rows per group
`.mean()`, `.median()`	average, middle value
`.sum()`, `.min()`, `.max()`	total, smallest, biggest
`.nunique()`	different values per group

Method	Gives
`.agg(["mean", "max"])`	several at once

6.5 The mean-of-booleans trick, with groupby

Combine Module 0.8 with groupby and you get a **rate per group**:

```
df.groupby("product")["timely"].mean() * 100
```

```
Card        100.0
Loan          0.0
Mortgage     50.0
```

"Percentage of complaints answered on time, per product." This is A line 129.

6.6 Grouping by several columns

```
df.groupby(["year", "product"]).size()
```

```
year  product
2023  Card        1
      Mortgage    1
2024  Card        2
      Loan        1
      Mortgage    1
```

The result has a **two-level index** (year and product). Most plotting tools prefer normal columns, so turn it back into a flat table:

```
df.groupby(["year", "product"]).size().reset_index(name="complaints")
```

```
   year   product  complaints
0  2023      Card           1
1  2023  Mortgage           1
2  2024      Card           2
...
```

`name=` names the new count column. This is A lines 139–141.

6.7 observed=True with category columns

When the grouping column is a `category`, pandas may create a group for **every possible category**, even ones with no rows (they'd show as 0 or NaN). `observed=True` keeps only groups that actually exist. Use it whenever you group by a category column (A lines 129, 140).

6.8 crosstab: a two-way count table

`crosstab` counts combinations of two columns: one as rows, one as columns.

```
pd.crosstab(df["product"], df["response"])
```

```
response  Explained  Money
product
Card              2      1
Loan              0      1
Mortgage          2      0
```

With `normalize="index"` each **row** becomes shares that add up to 1. Multiply by 100 for percentages:

```
pd.crosstab(df["product"], df["response"], normalize="index") * 100
```

```
response  Explained   Money
Card          66.7    33.3
Loan           0.0   100.0
Mortgage     100.0     0.0
```

Percentages let you compare products of very different sizes fairly. `normalize="columns"` does it per column, `normalize="all"` over the whole table. (A line 113.)

6.9 Sorting and picking columns of a result

```
mix = pd.crosstab(...)
mix = mix[["Money", "Explained"]]                    # choose and order
columns
mix = mix.sort_values("Money", ascending=False)      # sort rows by one
column
series.sort_values()                                  # sort a Series
smallest first
```

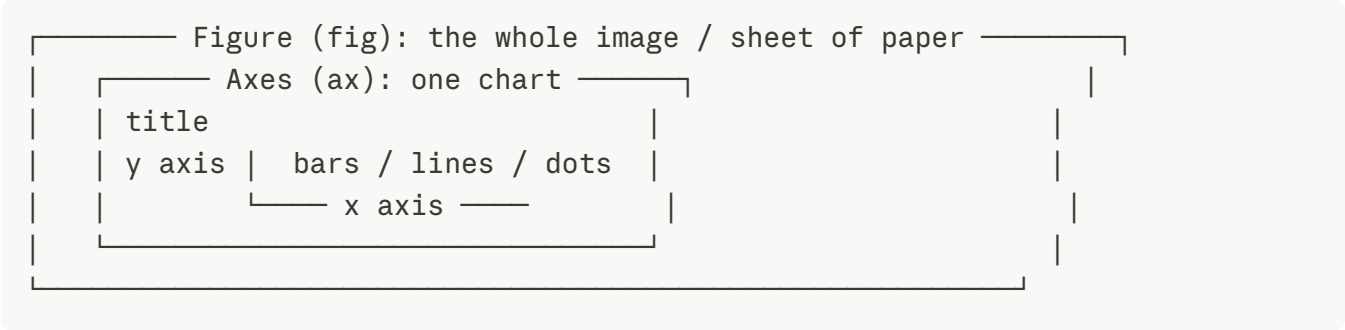
(A lines 116 and 40.)

Try it (6): With the table above: (a) the share of each response overall, (b) the median `days` per year, (c) for each product, the % of complaints that got "Money".

Module 7: matplotlib foundations

matplotlib draws charts from numbers you give it. Every chart is made of a **Figure** (the image) that holds one or more **Axes** (the chart areas). Learn to think in those two objects and the rest falls into place.

7.1 Figure vs Axes



- You **draw** and **label** on `ax`: `ax.plot`, `ax.barh`, `ax.set_title`.
- You **save** the `fig`: `fig.savefig`.

7.2 The basic recipe

Every chart in analysis.py follows these five steps:

```
fig, ax = plt.subplots(figsize=(8, 4))  # 1. create a figure with one chart,
8×4 inches
ax.plot([1, 2, 3], [10, 25, 15])          # 2. draw
ax.set_title("My first chart")           # 3. label
ax.set_xlabel("Month")
ax.set_ylabel("Complaints")
fig.savefig("first.png")                 # 4. save
plt.close(fig)                           # 5. free memory
```

Why `plt.close`? Each figure stays in memory until closed. A script that makes 20 charts without closing them can slow down or warn about too many open figures.

7.3 Chart types you use

Method	Chart	Data it needs	Used in
`ax.plot(x, y)`	line	x values and y values	A line 76 (monthly trend)
`ax.barh(labels, lengths)`	horizontal bars	names and numbers	A line 42 (`barh` helper)
`ax.bar(labels, heights)`	vertical bars	names and numbers	—
`ax.scatter(x, y)`	dots	two number columns	—
`ax.hist(values, bins=30)`	histogram	one number column	—

Horizontal or vertical bars? Use horizontal (`barh`) when labels are long, like product or company names. They stay readable instead of overlapping.

7.4 Feeding pandas data to matplotlib

A Series from `value_counts()` already has what a bar chart needs:

```
counts = df["product"].value_counts().sort_values()   # smallest first
fig, ax = plt.subplots()
ax.barh(counts.index.astype(str), counts.values)
```

- `counts.index` → bar labels; `.astype(str)` makes sure they're plain text, not categories.
- `counts.values` → bar lengths.
- `sort_values()` puts the smallest first, and `barh` draws the first bar at the **bottom**, so the biggest ends up on top.

For a line over time, the index holds the dates: `ax.plot(monthly.index, monthly.values)` (A line 76).

7.5 Sizing the figure

`figsize=(width, height)` in inches. A smart trick from A line 41: make the height depend on the number of bars so they never get squashed.

```
fig, ax = plt.subplots(figsize=(9, 0.42 * len(s) + 1.2))
```

7.6 Saving

```
fig.savefig("chart.png", dpi=150, bbox_inches="tight")
```

- `dpi` = resolution (dots per inch). 150 is sharp on screen; use 300 for print.
- `bbox_inches="tight"` trims extra white margin.
- The extension picks the format: `.png`, `.pdf`, `.svg`.

analysis.py sets `dpi` and `bbox` once for all charts with `rcParams` (Module 8.1), so `savefig` only needs the file name.

7.7 Saving to a folder with Path

```
from pathlib import Path
OUT = Path("plots")
OUT.mkdir(exist_ok=True)          # create the folder; fine if it exists
fig.savefig(OUT / "chart.png")    # / joins folder and file name →
plots/chart.png
```

(A lines 16–17, 50.)

7.8 Showing vs saving: backends

A **backend** is how matplotlib outputs a chart.

- In Jupyter, charts show automatically below the cell.
- In a script, `plt.show()` opens a window.
- On a server with no screen, there's no window to open. `matplotlib.use("Agg")` (A line 9) switches to a file-only backend. It must run **before** `import matplotlib.pyplot`.

7.9 Two styles of matplotlib code

You'll see both online. analysis.py uses the second one, which scales better.

```
# 1. "pyplot" style: quick, but hidden state
plt.plot(x, y)
plt.title("...")

# 2. "object" style: you hold fig and ax explicitly
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("...")       # note: set_title, not title
```

The names differ slightly: `plt.title` ↔ `ax.set_title`, `plt.xlabel` ↔ `ax.set_xlabel`, `plt.xlim` ↔ `ax.set_xlim`.

Try it (7): Make a horizontal bar chart of complaints per `channel`, biggest on top, and save it as `plots/my_channels.png`.

Module 8: matplotlib customization

The default chart is correct but plain. These tools make it readable: clean number formats, value labels, annotations, consistent colors, and several charts in one image.

8.1 Global style with rcParams

`plt.rcParams` is a dictionary of default settings. Change it once at the top and every later chart follows.

```
plt.rcParams.update({
    "axes.spines.top": False,     # remove top border line
    "axes.spines.right": False,   # remove right border line
    "axes.titleweight": "bold",
    "axes.titlelocation": "left",
    "savefig.dpi": 150,
    "savefig.bbox": "tight",
})
```

A lines 26–34 set background colors, text colors, grid color, title style and save settings this way. The benefit: one place to change the look of all 10 charts.

Keys are grouped by prefix: `figure.*` (the image), `axes.*` (the chart area), `xtick.*` / `ytick.*` (tick labels), `grid.*`, `savefig.*`, `font.*`.

8.2 Colors

Any matplotlib color can be a name (`"steelblue"`) or a **hex code** `"#RRGGBB"` : two hex digits each for red, green, blue, from `00` (none) to `ff` (full).

```
BLUE = "#2a78d6"
ax.barh(labels, values, color=BLUE)
```

Storing colors in variables (A lines 20–21) keeps them consistent across charts.

A **colormap** is a smooth range of colors used to show values (light = small, dark = big). You can build your own from a list:

```
from matplotlib.colors import LinearSegmentedColormap
BLUES = LinearSegmentedColormap.from_list("blues", ["#f4f8fd", "#3987e5",
"#0d366b"])
```

(A lines 22–23; used by the heatmap.) Built-in ones exist too: `"Blues"`, `"viridis"`.

8.3 Formatting axis numbers

By default the axis shows `3000000`. A **formatter** decides how each tick number is displayed.

```
import matplotlib.ticker as mtick

millions = mtick.FuncFormatter(lambda x, _: f"{x/1e6:.1f}M" if x >= 1e6 else
f"{x/1e3:.0f}k")
ax.yaxis.set_major_formatter(millions)     # 3000000 → 3.0M, 500000 → 500k

ax.xaxis.set_major_formatter(mtick.PercentFormatter(decimals=0))   # 95 → 95%
```

- `FuncFormatter` takes any function `(value, position) → text`. Here a `lambda` (Module 0.6).
- `PercentFormatter` adds `%`. It expects numbers already on a 0–100 scale by default.
- `ax.xaxis` for the horizontal axis, `ax.yaxis` for the vertical one.

(A lines 35, 45, 79, 131.)

8.4 Value labels on bars

`ax.text(x, y, "text")` writes text at a data position. For horizontal bars, bar number `i` sits at height `i`, and its end is at `x = value`:

```
for y, v in enumerate(s.values):
    ax.text(v, y, f"  {v:,.0f}", va="center", fontsize=9)
ax.set_xlim(0, s.max() * 1.15)   # 15% extra room so labels aren't cut off
```

- `va="center"`: vertical alignment, centered on the bar. (`ha` = horizontal alignment: `"left"`, `"center"`, `"right"`.)
- The two spaces in `" "` keep the text off the bar.

(A lines 47–49.)

8.5 Annotations with arrows

`ax.annotate` writes text and draws a line from it to a point:

```
ax.annotate("Peak: 450,000",
            xy=(peak_date, peak_value),          # the point to mark
            xytext=(-110, -10),                  # where the text goes...
            textcoords="offset points",          # ...measured in points from
xy
            arrowprops=dict(arrowstyle="-", lw=0.8))   # "-" = plain line, "-
>" = arrow
```

`"offset points"` means the text position is relative to the point, so it stays near the peak whatever the data scale. (A lines 82–84.)

8.6 Grids, limits and tick labels

```
ax.grid(axis="y", visible=False)    # hide horizontal grid lines (they'd
cross bars)
ax.set_xlim(0, 100)                 # fix the axis range
ax.set_xticklabels(ax.get_xticklabels(), rotation=25, ha="right")   # tilt
long labels
```

Rule of thumb: keep grid lines **along the value axis only**. On horizontal bars that means vertical lines, so hide the y grid (A line 46). On a line chart over time, hide the x grid (A line 80).

8.7 Several charts in one figure

`plt.subplots(rows, cols)` returns a **grid of Axes**:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes[0, 0].plot(...)        # top-left
axes[1, 1].barh(...)        # bottom-right

for ax, col in zip(axes.flat, ["product", "state", "channel", "response"]):
    s = df[col].value_counts().head(10).sort_values()
    ax.barh(s.index.astype(str), s.values)
    ax.set_title(col)

fig.suptitle("Overview", fontweight="bold")   # one title above all charts
fig.tight_layout()                            # stop charts overlapping
```

`axes.flat` turns the 2×2 grid into a simple list of 4 so you can loop over it.

8.8 Wrapping a chart in a function

When you draw the same kind of chart many times, put it in a function with parameters for what changes. That's the `barh()` helper (A lines 38–52): data, title, axis label, file name, and two formatting options with **default values** (`fmt=millions`) that callers can override (A line 131).

```
def barh(series, title, xlabel, fname, fmt=millions, label_fmt="{:,.0f}"):
    ...
barh(counts, "Title", "Complaints", "x.png")                    # uses
defaults
barh(rates, "Rate", "%", "y.png", fmt=mtick.PercentFormatter())  # overrides
one
```

Combine it with a loop over a settings list to make many charts at once:

```
CHARTS = [("product", None, "Complaints by product"),
          ("state", 15, "Top 15 states")]
for col, top_n, title in CHARTS:
    counts = df[col].value_counts()
    if top_n:
        counts = counts.head(top_n)
    barh(counts, title, "Complaints", f"{col}.png")
```

Try it (8): Take your chart from Try it 7. Add value labels, a `millions` formatter on the x axis, remove the y grid, and annotate the biggest bar with the text "Most used".

Module 9: seaborn

seaborn is built on top of matplotlib. You give it a **DataFrame and column names**, and it handles grouping, colors and legends for you. Anything seaborn draws is still a matplotlib chart, so everything from Modules 7–8 still works on it.

9.1 Theme

```
import seaborn as sns
sns.set_theme(style="whitegrid", font_scale=1.0)
```

One line restyles **all** later charts, including plain matplotlib ones. Styles: `"whitegrid"`, `"darkgrid"`, `"white"`, `"ticks"`. `font_scale=1.2` makes all text 20% bigger. (A line 25, followed by your own `rcParams` tweaks on top.)

9.2 The seaborn way: data + column names

matplotlib wants the numbers themselves. seaborn wants the **table** and the **names of the columns** to use:

```
# matplotlib
ax.plot(yearly["year"], yearly["complaints"])

# seaborn
sns.lineplot(data=yearly, x="year", y="complaints", hue="product")
```

The seaborn line draws **one line per product**, each in its own color, with a legend, from one call. In matplotlib you'd write a loop.

That's why seaborn prefers **"long" tables**: one row per observation, with columns like `year | product | complaints`. This is exactly what `groupby(...).size().reset_index(name="complaints")` produces (Module 6.6).

9.3 The arguments you'll use most

Argument	Meaning
`data=`	the DataFrame
`x=`, `y=`	column names for the axes
`hue=`	column that decides the **color** (one color per value)
`palette=`	which colors: a name (`"Set2"`) or a dict `{value: color}`
`col=`, `row=`	column that splits the data into **separate small charts** (figure-level only)
`ax=`	draw on this matplotlib Axes (axes-level only)

9.4 Two kinds of seaborn functions

This distinction confuses most beginners, and your script uses both.

	Axes-level	Figure-level
Examples	`heatmap`, `lineplot`, `barplot`, `scatterplot`	`relplot`, `catplot`, `displot`
Draws on	one `ax` you pass with `ax=ax`	its own new figure

	Axes-level	Figure-level
Can make a grid of charts?	no	yes, with `col=` / `row=`
Returns	an `ax`	a grid object `g`
Size set with	`plt.subplots(figsize=...)`	`height=` and `aspect=`
Save with	`fig.savefig(...)`	`g.savefig(...)`

Chart 6 (heatmap) is axes-level; chart 9 (relplot) is figure-level.

9.5 heatmap

A heatmap colors each cell of a table by its value. Give it a DataFrame whose rows and columns are already the categories, like a `crosstab` (Module 6.8).

```
mix = pd.crosstab(df["product"], df["response"], normalize="index") * 100
fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(mix,
            annot=True,        # write the value in each cell
            fmt=".1f",         # with 1 decimal
            cmap="Blues",      # colormap (A uses the custom BLUES)
            linewidths=2,      # gaps between cells
            cbar_kws={"label": "% of the product's complaints"},  # color-bar
legend title
            ax=ax)
ax.set_xlabel("")
fig.savefig("heatmap.png")
```

`fmt` uses the same codes as f-strings without the colon: `".1f"`, `",.0f"`, `"d"` for whole numbers. (A lines 113–126.)

9.6 relplot and facets

`relplot` draws relationships (lines or dots) and can split them into a **grid of small charts**, one per value of a column. These are called **facets** or **small multiples**.

```
g = sns.relplot(data=yearly, x="year", y="complaints",
                col="product", col_wrap=2,     # one chart per product, 2 per
row
                kind="line", marker="o",       # lines with a dot per year
                hue="product", legend=False,   # color by product; titles
already say which
                facet_kws={"sharey": False},   # each chart gets its own y
scale
                height=3.2, aspect=1.6)        # each chart: 3.2 in tall,
1.6× as wide
```

`sharey` : shared or separate scales?

- Shared (`True` , the default): easy to compare sizes between charts, but small series look flat.
- Separate (`False`): each chart shows its own trend clearly, but you must read the axis numbers before comparing.

A uses separate scales because credit reporting is far bigger than the other products. Pick based on the question: "which is bigger?" → shared; "how did each one change?" → separate.

`kind="scatter"` gives dots instead of lines.

9.7 Working with the grid object g

```
g.set_titles("{col_name}")           # each small chart titled with its
product
g.set_axis_labels("", "Complaints")   # x label, y label for all charts
for ax in g.axes.flat:                # reach each matplotlib Axes inside
    ax.yaxis.set_major_formatter(millions)
g.figure.suptitle("Complaints per year", y=1.03)   # one title above the grid
g.savefig("yearly.png")
plt.close(g.figure)
```

`g.axes.flat` is how you get back to plain matplotlib to customize each chart. (A lines 148–155.)

9.8 Category columns and seaborn

If the `hue` or `col` column is a pandas `category`, seaborn may make room for **every category**, including ones you filtered out, giving empty panels. Converting to text first avoids that:

```
yearly["product"] = yearly["product"].astype(str)
```

(A line 142.)

9.9 Other seaborn charts worth knowing

Function	Shows	Example question
`sns.barplot`	average per category, with an error bar	average days per product?
`sns.countplot`	count per category	complaints per channel?
`sns.histplot`	distribution of one number	how are `days_to_company` spread?
`sns.boxplot`	median and spread per category	which product has the slowest forwarding?
`sns.catplot`	figure-level version of the above, with `col=`	the same, split by year

Try it (9): Make a heatmap of complaints per `year` (rows) × `channel` (columns), as % of each year. Then a `relplot` of yearly complaints per channel with one panel per channel.

Module 10: Capstone exercises and answers

The capstone uses your real `complaints_clean.csv`. Load it the same way as analysis.py does, then work through the tasks in order. Each one combines several modules.

10.1 Capstone tasks

- [ ] **Task 1: Fastest and slowest companies.** Among companies with at least 1,000 complaints, find the 10 with the highest median `days_to_company`. Draw them with your own bar chart. *Modules 3, 6, 8.*

- [ ] **Task 2: Monetary relief rate per year.** For each year, the % of complaints that ended "Closed with monetary relief". Draw it as a line chart with a % axis. *Modules 0.8, 5, 6, 8.*
- [ ] **Task 3: State heatmap.** For the top 10 states, a heatmap of the response mix (% of each state's complaints). *Modules 6.8, 9.5.*
- [ ] **Task 4: Fix the partial-year problem.** Chart 9 in analysis.py shows the current year as a drop. Remove incomplete years before grouping, then redraw it. *Module 5.4.*
- [ ] **Task 5: Automatic overview.** Write a loop that makes one 2×3 image with the top-10 bar chart for six category columns. *Modules 2.3, 8.7.*
- [ ] **Task 6: Your own question.** Pick a question about the data you're curious about, answer it with one number and one chart, and write one sentence explaining what it shows.

Hints:

- Task 1: `counts = df["company"].value_counts()`, then `big = counts[counts >= 1000].index`, filter with `isin(big)`, then `groupby("company", observed=True)["days_to_company"].median()`, `.sort_values().tail(10)`.
- Task 2: make a boolean column first: `df["money"] = df["response"] == "Closed with monetary relief"`, then `groupby("year")["money"].mean() * 100`.
- Task 3: `top = df["state"].value_counts().head(10).index`, filter, then `pd.crosstab(..., normalize="index") * 100`. If the empty categories show up, add `.astype(str)` on both columns before the crosstab.
- Task 4: the last full year is `df["date_received"].max().year - 1` if the max date isn't 31 December.
- Task 5: `cols = df.select_dtypes("category").columns[:6]`, `fig, axes = plt.subplots(2, 3, figsize=(18, 10))`, `zip(axes.flat, cols)`.

10.2 Answers to the "Try it" exercises

Try it (0)

```
print(f"{1234.6:,.0f}")                          # 1,235
dict(zip(["a", "b", "c"], [1, 2, 3]))          # {'a': 1, 'b': 2, 'c': 3}
```

Careful: `f"{1234.5:,.0f}"` gives `1,234`, not `1,235`. Python rounds exact halves to the nearest even number.

Try it (1)

```
df["timely"].mean()      # 0.75
```

Try it (2)

```
small = pd.read_csv("complaints.csv", usecols=["Product", "State"],
nrows=1000)
print(small.memory_usage(deep=True).sum())
print(small.astype("category").memory_usage(deep=True).sum())
```

Try it (3)

```
df[df["state"].isin(["NY", "CA"]) & (df["days"] == 0)]    # only EXPERIAN, CA
```

Try it (4)

```
df = raw.rename(columns={...})
df["date_received"] = pd.to_datetime(df["date_received"], errors="coerce")
df = df.dropna(subset=["date_received", "company"])      # removes 102 and
103
df["company"] = df["company"].str.strip().str.upper()
df = df.drop_duplicates(subset="complaint_id")           # removes the 2nd
101
```

Try it (5)

```
df["date_received"].dt.day_name().value_counts()
df.groupby("year")["days_to_company"].median().idxmax()
```

Try it (6)

```
df["response"].value_counts(normalize=True)                          # (a)
df.groupby("year")["days"].median()                                  # (b)
(df["response"] == "Money").groupby(df["product"]).mean() * 100      # (c)
```

For (c), you can also use `pd.crosstab(df["product"], df["response"], normalize="index")["Money"] * 100`.

Try it (7)

```
s = df["channel"].value_counts().sort_values()
fig, ax = plt.subplots(figsize=(8, 4))
ax.barh(s.index.astype(str), s.values)
ax.set_title("Complaints by channel")
fig.savefig("plots/my_channels.png")
plt.close(fig)
```

Try it (8)

```
for y, v in enumerate(s.values):
    ax.text(v, y, f"  {v:,.0f}", va="center")
ax.xaxis.set_major_formatter(millions)
ax.grid(axis="y", visible=False)
ax.set_xlim(0, s.max() * 1.15)
ax.annotate("Most used", (s.max(), len(s) - 1), xytext=(-80, -20),
            textcoords="offset points", arrowprops=dict(arrowstyle="->"))
```

The biggest bar is the last one (sorted smallest first), so it sits at `y = len(s) - 1`.

Try it (9)

```
mix = pd.crosstab(df["year"], df["channel"], normalize="index") * 100
fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(mix, annot=True, fmt=".0f", cmap="Blues", ax=ax)
fig.savefig("plots/year_channel.png")

yearly = df.groupby(["year", "channel"],
observed=True).size().reset_index(name="complaints")
yearly["channel"] = yearly["channel"].astype(str)
g = sns.relplot(data=yearly, x="year", y="complaints", col="channel",
col_wrap=3,
                kind="line", marker="o", facet_kws={"sharey": False},
height=3)
g.savefig("plots/yearly_channel.png")
```

Cheat sheet

Every pattern from the course on one page. The last column says where your scripts use it (C = clean.py, A = analysis.py).

pandas

I want to…	Code	Where
load a CSV	`pd.read_csv("f.csv", usecols=[...], dtype={...}, parse_dates=[...])`	C 91, A 58
load in chunks	`for chunk in pd.read_csv("f.csv", chunksize=1_000_000):`	C 91–92
look at the data	`df.shape`, `df.dtypes`, `df.head()`, `df.describe(include="all").T`	A 64–67
rename columns	`df.rename(columns={"Old": "new"})`	C 55
text → date / number	`pd.to_datetime(s, errors="coerce")`, `pd.to_numeric(s, errors="coerce")`	C 58–59, 80
drop rows with blanks	`df.dropna(subset=["a", "b"])`	C 60
fill blanks	`s.fillna("Unknown")`	C 70–72
clean text	`s.str.strip()`, `s.str.upper()`	C 65, 68
merge category names	`s.replace({"old": "new"})`	C 67
Yes/No → True/False	`s.eq("Yes")`	C 75
whole numbers with blanks	`s.astype("Int16")`	C 78
remove duplicates	`df.drop_duplicates(subset="id")`	C 97
filter rows	`df[df["a"] > 5]`, `df[(cond1) & (cond2)]`	A 71
"is one of" / "is not one of"	`df[s.isin(lst)]`, `df[~s.isin(lst)]`	C 98, A 100
rows + a column	`df.loc[mask, "col"]`	A 100
count each value	`s.value_counts()`, `.head(10)`	A 90–134
stat per group	`df.groupby("g")["col"].mean()`	A 129
% True per group	`df.groupby("g")["bool_col"].mean() * 100`	A 129
count per 2 groups, flat	`df.groupby(["a", "b"]).size().reset_index(name="n")`	A 139–141

I want to…	Code	Where
two-way % table	`pd.crosstab(df["a"], df["b"], normalize="index") * 100`	A 113
count per month	`df.set_index("date").resample("MS").size()`	A 74
date parts	`s.dt.year`, `s.dt.to_period("M")`, `(d2 - d1).dt.days`	C 78–79, A 71
where is the max	`s.idxmax()`	A 81
save a CSV	`df.to_csv("f.csv", index=False)`; chunks: `mode="a", header=(i == 1)`	C 102
save memory	`dtype="category"` for few-value text; `int16` / `Int16`	A 61

matplotlib

I want to…	Code	Where
new chart	`fig, ax = plt.subplots(figsize=(w, h))`	A 41
grid of charts	`fig, axes = plt.subplots(2, 2)`; loop `axes.flat`	—
line / bars	`ax.plot(x, y)`, `ax.barh(labels, values)`	A 76, 42
title and labels	`ax.set_title()`, `ax.set_xlabel()`, `ax.set_ylabel()`	A 43–44
format axis numbers	`ax.yaxis.set_major_formatter(mtick.FuncFormatter(f))`	A 79
percent axis	`mtick.PercentFormatter(decimals=0)`	A 131
value labels	`ax.text(x, y, "text", va="center")`	A 48
point at something	`ax.annotate("txt", (x, y), xytext=(dx, dy), textcoords="offset points", arrowprops=dict(arrowstyle="-"))`	A 82
hide grid lines	`ax.grid(axis="y", visible=False)`	A 46
axis range	`ax.set_xlim(0, top)`	A 49

I want to…	Code	Where
tilt labels	`ax.set_xticklabels(ax.get_xticklabels(), rotation=25, ha="right")`	A 123
global style	`plt.rcParams.update({...})`	A 26
save and free	`fig.savefig(path)`, `plt.close(fig)`	A 50–51
no window (server)	`matplotlib.use("Agg")` before importing pyplot	A 9

seaborn

I want to…	Code	Where
nicer defaults	`sns.set_theme(style="whitegrid")`	A 25
heatmap of a table	`sns.heatmap(df, annot=True, fmt=".1f", cmap="Blues", ax=ax)`	A 118
one line per group	`sns.lineplot(data=df, x="a", y="b", hue="g")`	—
one small chart per group	`sns.relplot(data=df, x="a", y="b", col="g", col_wrap=2, kind="line")`	A 144
separate y scales	`facet_kws={"sharey": False}`	A 147
customize each panel	`for ax in g.axes.flat:`	A 150
title over the grid	`g.figure.suptitle("...")`	A 152

Five habits that prevent most bugs

1. **Reassign:** `df = df.dropna()`. Most methods return a new object and leave the original unchanged.
2. **Parentheses around each condition** when combining with `&`, `|`.
3. **Check after every step:** `print(df.shape)` and `df.head()` to see what changed.
4. **Use** `errors="coerce"` on real-world data, then count what became missing with `.isna().sum()`.
5. `observed=True` whenever you group by a category column.